

Programmmentwurf: R-Paket `poisselect`

Poisson-Selektionsmodell für Zähldaten, Hausarbeit Advanced Data Analysis with R

Sebastian Jung, Finn Metzler

2026-09-03

1 Vorgehen

Der Entwurf folgt dem Top-Down-Workflow der Vorlesung: zuerst die Aufgabe mit Input und Output definieren, dann in Teilaufgaben zerlegen, die Reihenfolge festlegen, Gemeinsamkeiten herausziehen (DRY) und für jede Teilaufgabe den Pseudocode schreiben. Kommentarzeilen beschreiben die Schritte auf Entwurfsebene, es ist kein lauffähiger Code.

2 Aufgabe definieren: Hauptfunktion

```
# Input:
#   outcome    zweiseitige Formel  y ~ x1 + x2          (Zaehlvariable y,
#               nur fuer s = 1 beobachtet, sonst NA)
#   selection   zweiseitige Formel  s ~ x1 + z1          (0/1-Indikator s,
#               z1 = Exclusion Restriction)
#   data        data.frame mit allen Variablen beider Formeln
#   K           Anzahl der Gauss-Hermite-Knoten, Default 20
#   start       optionale Startwerte c(beta, gamma, sigma, rho) oder NULL
#   control     benannte Liste mit Steuerparametern fuer optim()
#
# Output:
#   S3-Objekt der Klasse "poisselect" mit beta, gamma, sigma, rho,
#   Standardfehlern, Kovarianzmatrix, Log-Likelihood, Konvergenzstatus,
#   Fallzahlen (n, davon selektiert) und call

poisselect <- function(outcome, selection, data, K = 20,
                       start = NULL, control = list()) {
  # Unterfunktionen in fester Reihenfolge, siehe naechster Abschnitt
  output
}
```

3 Reihenfolge der Teilaufgaben

```
poisselect <- function(outcome, selection, data, K = 20,
                       start = NULL, control = list()) {
  check_arguments(outcome, selection, data, K, start, control)
  # 1. Pruefe die Argumente, bevor Daten angefasst werden (fail fast)
  model <- build_model_data(outcome, selection, data)
  # 2. Formeln + data.frame -> y, s, Designmatrizen x, z,
  #    einmalige Aufteilung in selektiert / nicht selektiert
  check_model_data(model)
  # 3. Pruefe die abgeleiteten Daten: Wertebereiche, NA-Muster,
  #    Fallzahlen, Rang, Exclusion Restriction (fail appropriately)
  check_start_values(start, model)
```

```

# 4. Erst jetzt pruefbar: Laenge von 'start' haengt von x und z ab
quadrature <- build_gauss_hermite(K)
# 5. Knoten t_k und Gewichte w_k zur Gewichtsfunktion exp(-t^2)
theta_start <- compute_start_values(model, start)
# 6. Startvektor auf der unrestringierten Skala
optimum <- maximise_loglik(theta_start, model, quadrature, control)
# 7. Maximierung mit optim(), BFGS, analytischer Gradient
uncertainty <- compute_standard_errors(optimum$par, model, quadrature)
# 8. Hesse-Matrix am Optimum, Inverse, Delta-Methode
new_poisselect(optimum, uncertainty, model, quadrature, match.call())
# 9. S3-Objekt zusammenbauen
}

```

Pruefen, Aufbereiten, Pruefen, Quadratur, Startwerte, Optimieren, Standardfehler, Objekt bauen.

4 Pseudocode der Unterfunktionen

4.1 Argument-Checks (check_input.R)

```

# Input:  die Argumente von poisselect()
# Output: invisible(TRUE); wirft informative Fehler (call. = FALSE)
check_arguments <- function(outcome, selection, data, n_nodes, start,
                             control) {
  stop, falls data fehlt
  check_formula(outcome), check_formula(selection)
  # Formel, zweiseitig, ohne '.' (sonst landet die Response der
  # anderen Gleichung unter den Kovariablen)
  assert: data ist ein data.frame mit >= 1 Zeile
  assert: alle Variablen beider Formeln kommen in data vor (alle fehlenden
  auf einmal nennen)
  assert: K ist eine ganze Zahl in [2, 200]
  # K = 1: der einzige Knoten liegt bei 0, sigma faellt heraus
  assert: start ist NULL oder numerisch, endlich, ohne NA
  assert: control ist eine Liste, Namen Teilmenge der optim-Controls
}

# Input:  model (abgeleitete Daten)
# Output: invisible(TRUE); Fehler bzw. Warnung
check_model_data <- function(model) {
  s: keine NA, nur 0/1, beide Gruppen besetzt
  y auf s == 1: numerisch, keine NA, endlich, >= 0, ganzzahlig,
  nicht alle 0; y bei s == 0 beobachtet -> message
  Fallzahl: n > Anzahl Parameter, n_selected > n_beta + 1
  x auf selektierten Zeilen und z ueberall: endlich, voller Spaltenrang
  (kollineare Spalten beim Namen nennen)
  warn: keine Exclusion Restriction (z-Spalten ohne Intercept sind
  Teilmenge der x-Spalten)
}

# Input:  start, model
# Output: invisible(TRUE)
check_start_values <- function(start, model) {
  Laenge n_beta + n_gamma + 2, sigma-Eintrag > 0, |rho-Eintrag| < 1
}

```

4.2 Daten aufbereiten (model_data.R)

```
# Input:  outcome, selection, data
# Output: Liste mit y, s, selected, x, z, x_selected, z_selected,
#         z_unselected, y_selected, terms, xlevels, n, n_selected,
#         n_beta, n_gamma
build_model_data <- function(outcome, selection, data) {
  fuer beide Gleichungen dieselbe Hilfsfunktion (DRY):
  build_equation_data(formula, data)
  # model.frame mit na.action = na.pass (NA in y ist erwuenscht),
  # model.matrix, terms, xlevels (fuer predict mit newdata)
  s <- as_selection_indicator(response)
  # logisch -> 0/1; Faktor/character mit Hinweis ablehnen
  Aufteilung in selektierte und nicht-selektierte Zeilen EINMAL hier,
  nicht bei jeder Likelihood-Auswertung
}
```

4.3 Gauss-Hermite-Quadratur (gauss_hermite.R)

```
# Input:  n_nodes (K)
# Output: Liste mit nodes (aufsteigend) und weights
build_gauss_hermite <- function(n_nodes) {
  Jacobi-Matrix der monischen Hermite-Rekursion:
  Diagonale 0, Nebendiagonale sqrt(k / 2), k = 1, ..., K - 1
  (obere Nebendiagonale fuellen, dann jacobi + t(jacobi))
  Eigenzerlegung (symmetrisch)
  nodes <- Eigenwerte, aufsteigend sortiert
  weights <- sqrt(pi) * (erste Komponente des Eigenvektors)^2
}
```

4.4 Log-Likelihood und Gradient (loglik.R)

```
# Input:  theta = [beta, gamma, log(sigma), atanh(rho)], model, quadrature
# Output: Skalar, approximierte Log-Likelihood
compute_loglik <- function(theta, model, quadrature) {
  parameters <- split_parameters(theta) # exp(), tanh() zurueck
  pieces <- compute_node_pieces(parameters, model, quadrature)
  # n_sel x K Matrizen (DRY: auch Gradient und Plot 1 nutzen sie)
  # log_rate[i, k] = x_i'beta + sqrt(2) sigma t_k
  # eta[i, k]      = (z_i'gamma + sqrt(2) rho t_k) / sqrt(1 - rho^2)
  # log_selection  = pnorm(eta, log.p = TRUE)
  # log_weight[k]  = log(w_k) - log(pi) / 2
  a <- compute_log_terms(y_selected, pieces)
  # a[i, k] = log_weight[k] + dpois(y_i, exp(log_rate[i, k]), log = TRUE)
  #         + log_selection[i, k]
  ll_selected <- sum(log_sum_exp_rows(a))
  # A_i = max_k a[i, k]; A_i + log(sum_k exp(a[i, k] - A_i))
  ll_unselected <- sum(pnorm(z_unselected %*% gamma,
    lower.tail = FALSE, log.p = TRUE))
  # ln(1 - Phi) direkt als oberes Ende, nie log(1 - pnorm(.))
  ll_selected + ll_unselected
}

# Input:  theta, model, quadrature
# Output: Vektor gleicher Laenge wie theta
compute_loglik_gradient <- function(theta, model, quadrature) {
  dieselben pieces und a wie oben
  p[i, k] <- exp(a[i, k] - LSE_i) # Gewicht des Knotens k
}
```

```

Ableitungen je Knoten:
  nach beta: (y_i - mu_ik) * x_i
  nach sigma: (y_i - mu_ik) * sqrt(2) t_k
  nach gamma: phi(eta)/Phi(eta) * z_i / sqrt(1 - rho^2)
  nach rho: phi(eta)/Phi(eta) * (sqrt(2) t_k + rho z_i'gamma)
            / (1 - rho^2)^(3/2)
mit p gewichtet ueber k summieren, ueber i aufsummieren;
nicht-selektierte Einheiten: -phi(z'gamma)/(1 - Phi(z'gamma)) * z_i
Kettenregel: Eintrag sigma * sigma, Eintrag rho * (1 - rho^2)
}

# Zielfunktion fuer optim(): -loglik; nicht-endlich -> Strafe 1e10
compute_negative_loglik(), compute_negative_gradient()

```

4.5 Reparametrisierung (parameters.R)

```

# Restriktionen sigma > 0 und -1 < rho < 1:
# optim() arbeitet unrestringiert auf
#   theta = [ beta (n_beta), gamma (n_gamma), log(sigma), atanh(rho) ]
# split_parameters(theta, n_beta, n_gamma) -> Liste beta, gamma, sigma, rho
# pack_parameters(beta, gamma, sigma, rho) -> theta
# build_parameter_names(model) -> "outcome_x1", ..., "selection_z1",
#                                   "sigma", "rho"

```

4.6 Startwerte (start_values.R)

```

# Input: model, start (oder NULL)
# Output: theta_start auf der unrestringierten Skala
compute_start_values <- function(model, start) {
  falls start gegeben: pack_parameters(start) und fertig
  gamma_0 <- Probit-GLM s ~ z auf allen Zeilen
  # Devianz numerisch 0 -> Warnung "perfekte Trennung"
  beta_0 <- Poisson-GLM y ~ x auf den selektierten Zeilen
  # glm.fit mit suppressWarnings, nicht-endliche Koeffizienten -> 0
  sigma_0 <- Momentenschaetzer aus der Ueberdispersion
  # Var(Y) = mu + mu^2 (exp(sigma^2) - 1), auf [0.1, 1.55] begrenzt
  beta_0[Intercept] <- beta_0[Intercept] - sigma_0^2 / 2
  # das GLM schaezt log E[Y|x] = x'beta + sigma^2 / 2
  rho_0 <- 0 # ignorierbare Selektion
  pack_parameters(beta_0, gamma_0, sigma_0, rho_0)
}

```

4.7 Maximierung (optimise.R)

```

# Input: theta_start, model, quadrature, control
# Output: Liste von optim()
maximise_loglik <- function(theta_start, model, quadrature, control) {
  control <- Defaults maxit = 1000, reltol = 1e-10, durch Nutzer ueberschreibbar
  optim(theta_start, fn = compute_negative_loglik,
        gr = compute_negative_gradient, method = "BFGS")
  warn_about_fit(): Konvergenzcode != 0, |rho| > 0.99, sigma < 1e-3
}

```

4.8 Standardfehler (standard_errors.R)

```

# Input: theta (Optimum), model, quadrature
# Output: vcov und standard_errors auf der Originalskala

```

```
compute_standard_errors <- function(theta, model, quadrature) {
  H <- optimHess(theta, fn, gr)      # differenziert den analytischen
                                     # Gradienten numerisch
  V_u <- solve(H)                   # singulaer -> Warnung, NA-Matrix
  J   <- diag(c(1, ..., 1, sigma, 1 - rho^2))
                                     # d sigma / d log sigma = sigma
                                     # d rho / d atanh rho   = 1 - rho^2
  V   <- J V_u J' (= V_u * outer(diag(J), diag(J)))
  se  <- sqrt(diag(V)), nicht-positive Varianzen -> Warnung, NA
}
```

4.9 Konstruktor (new_poisselect.R)

```
# Input: optimum, uncertainty, model, quadrature, call
# Output: Objekt der Klasse "poisselect"
new_poisselect <- function(...) {
  list(call, coefficients = list(outcome, selection), sigma, rho,
        standard_errors = list(outcome, selection, sigma, rho), vcov,
        loglik, npar, aic, converged, convergence, counts, message,
        n, n_selected, K, quadrature, model, theta)
  class <- "poisselect"
}
```

5 S3-Methoden

```
print.poisselect(x, digits, ...)
  call, Fallzahlen (n, selektiert, nicht selektiert), Log-Likelihood,
  AIC, K, Konvergenz, Koeffizienten beider Gleichungen, sigma, rho;
  invisible(x)

summary.poisselect(object, ...)
  build_coefficient_table(estimate, se) fuer Outcome, Selektion und
  (sigma, rho): Estimate, Std. Error, z = Estimate/SE, p = 2 (1 - Phi(|z|))
  -> Objekt "summary.poisselect" mit rho, loglik, aic, Fallzahlen
print.summary.poisselect(x, ...)
  printCoefmat() je Tabelle, rho mit Richtung der Verzerrung im Klartext,
  Log-Likelihood, AIC, K, Konvergenz

plot.poisselect(x, which = 1:2, max_count = NULL, n_grid = 41, ...)
  Plot 1: beobachtete relative Haeufigkeiten vs. modell-implizierte
  Verteilung der selektierten Einheiten
  
$$\Pr(y_i = m \mid s_i = 1) = \frac{\sum_k (w_k / \sqrt{\pi}) p(m \mid \mu_{ik}) \Phi(\eta_{ik})}{\sum_k (w_k / \sqrt{\pi}) \Phi(\eta_{ik})}$$

  Nenner einmal, Zaehler je m, beides per Log-Sum-Exp;
  Mittel ueber die selektierten Einheiten; Support bis zum
  99 %-Quantil der Counts
  Plot 2: Log-Likelihood auf einem rho-Gitter in (-0.99, 0.99) inkl.
  Schaetzwert, uebrige Parameter am Optimum fixiert
  par() sichern und per on.exit() wiederherstellen

predict.poisselect(object, newdata = NULL, type, ...)
  Designmatrix: gespeichert, oder aus terms + xlevels neu gebaut
  (fehlende Variablen und NA in newdata beim Namen nennen)
  link      -> x'beta
  response  -> exp(x'beta + sigma^2 / 2)      # unbedingte Erwartung
  pselect   -> Phi(z'gamma)
```

```
coef.poisselect(object, which = c("all", "outcome", "selection"), ...)
vcov.poisselect(object, ...)          # volle Matrix, Originalskala
logLik.poisselect(object, ...)        # mit df, nobs fuer AIC()/BIC()
```

6 Datengenerator für Tests und Beispiele

```
# Input:  n, beta (3), gamma (3), sigma, rho
# Output: data.frame mit y (NA falls s = 0), s, x1, x2, z1, y_complete
simulate_poisselect <- function(n, beta, gamma, sigma, rho) {
  x1, x2, z1, u, e ~ N(0, 1) unabhaengig
  eps <- sigma * (rho * u + sqrt(1 - rho^2) * e)
  # Var(eps) = sigma^2, Var(u) = 1, Corr(eps, u) = rho
  s <- 1{gamma_0 + gamma_1 x1 + gamma_2 z1 + u > 0}
  y ~ Poisson(exp(beta_0 + beta_1 x1 + beta_2 x2 + eps))
  y beobachtet nur fuer s = 1
}
```

Damit lässt sich der Parameterrückgewinnungstest direkt umsetzen.

7 Zuordnung Entwurf zu Dateien

Datei in R/	Funktionen
poisselect.R	poisselect()
check_input.R	check_arguments(), check_formula(), check_variables_available(), check_start_values(), check_model_data() und die Einzelchecks
model_data.R	build_model_data(), build_equation_data(), as_selection_indicator()
gauss_hermite.R	build_gauss_hermite()
loglik.R	compute_loglik(), compute_node_pieces(), compute_log_terms(), add_log_weight(), log_sum_exp_rows(), compute_loglik_gradient(), compute_negative_loglik(), compute_negative_gradient()
parameters.R	split_parameters(), pack_parameters(), build_parameter_names()
start_values.R	compute_start_values(), fit_probit_start(), fit_glm_start(), estimate_start_sigma()
optimise.R	maximise_loglik(), warn_about_fit()
standard_errors.R	compute_standard_errors(), invert_information(), extract_standard_errors()
new_poisselect.R	new_poisselect()
methods_print.R	print.poisselect(), print_coefficient_block(), describe_convergence()
methods_summary.R	summary.poisselect(), build_coefficient_table(), print.summary.poisselect(), print_coefficient_table(), describe_selection_bias()

Datei in R/	Funktionen
methods_plot.R	plot.poisselect(), plot_count_distribution(), compute_count_distribution(), plot_rho_profile(), compute_rho_profile()
methods_predict.R	predict.poisselect(), build_prediction_matrix()
methods_extract.R	coef.poisselect(), vcov.poisselect(), logLik.poisselect()
simulate.R	simulate_poisselect(), check_simulate_arguments()
data.R	Dokumentation des Datensatzes <code>sim_selection</code>
poisselect-package.R	Paketdokumentation, @importFrom